

3 Alien Language

Task 3 Jupyter Notebook template is provided for you. Please rename the Jupyter Notebook as

Task_3_<class>_<index number>_<name>.ipynb

You are a cryptographer decoding messages sent by an alien civilization. The aliens use a special numeric encoding system where:

- Each character in their messages is first converted to its ASCII code.
- Then the ASCII code is converted to a 3 digits base-7 number (since their civilization uses base-7).

Your mission is to decode the secret messages back into human-readable text.

For example, the following alien code is received.

205166226166220203211216231216

The above alien code is made of 10 characters with each character encoded by 3 digits. The table below first divides the code into 10 parts with each part containing 3 digits. Then, it calculates its respective denary number. Finally, convert it back to ascii letter.

Code	205	166	226	166	220	203	211	216	231	216
Denary	103	97	118	97	112	101	106	111	120	111
ASCII	g	a	v	a	p	e	j	o	x	o

The decoded word is therefore gavapejoxo.

Task 3.1

Implement the function `display_alien_words(alien_codes)` which takes a list of alien code stings as input and display their corresponding decoded words in the given format.

For example,

```
>>> alien_codes_3 =
['205166226166220203211216231216',
 '224166214166220166200210',
 '200166220203226216']
>>> display_alien_words(alien_codes_3)
WORD      CODE
gavapejoxo 205166226166220203211216231216
tamapabi   224166214166220166200210
bapevo     200166220203226216
```

Test your function using `alien_codes` that is provided in the template.

[5]

[RVHS/2025/Q3 Cont']

Task 3.2

As a cryptographer decoding alien messages for many years, you have come to understand more alien words. These words are stored in the variable `known_alien_word_list` as a list in the template provided. You are often required to check if a decoded alien word is inside the known alien word list. Storing these words in a list makes this search operation $O(n)$ time complexity.

Implement the class `AlienKnowledgeDataBase` where its search operation can now be done in $O(\lg n)$ time complexity.

Class <code>AlienKnowledgeDataBase</code>	
Attributes	
You can decide on whatever attribute you need for this class	
Functions	
<code>__init__(known_alien_word_list)</code>	The constructor takes in the known alien word list and store the words in a data structure that allows the search operation to be done in $O(\lg n)$ time complexity.
<code>search(alien_word)</code>	This function checks if <code>alien_word</code> is one of the words that can be found in <code>known_alien_word_list</code> . It returns <code>True</code> if it is found and <code>False</code> otherwise. The search operation must be in $O(\lg n)$ time complexity.

Test your implementation using the `task3_2()` that is provided in the template. [4]

Task 3.3

Implement the class `AlienKnowledgeDataBase2` where its search operation can now be done in $O(1)$ time complexity.

Test your implementation using the `task3_3()` that is provided in the template. [5]